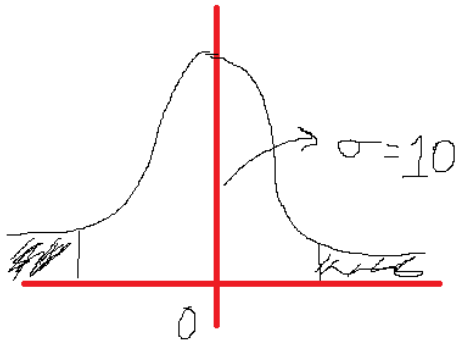


Averaging Filter on a Noisy dataset

Github (for code): https://github.com/pandaiscodingcpu/autonomous_navigation

Assumptions made in the synthetic dataset:

1. The dataset consists of 2000 readings at a rate of 1Hz.
2. External noise is added with mean: 0 and Standard deviation (S.D/sigma): 10



****The noise follows a Normal Distribution ($\mu = 0$ and $\sigma = 10$)**

3. Since, the data here is synthetic the perfect effect of oversampling cannot be demonstrated without adding complexity. Hence, oversampling is not demonstrated.
4. Idea is to just demonstrate the difference between Raw noisy readings and averaged out readings. (Code is not included due to the limitations of the scope of this document, only results are shown)

An Important thing to understand:

Median

dataset =

Pkt	Alt
1	x_1
2	x_2
3	x_3
...	...
2000	x_{2000}

Working with only [Alt]
Pkt. Col.

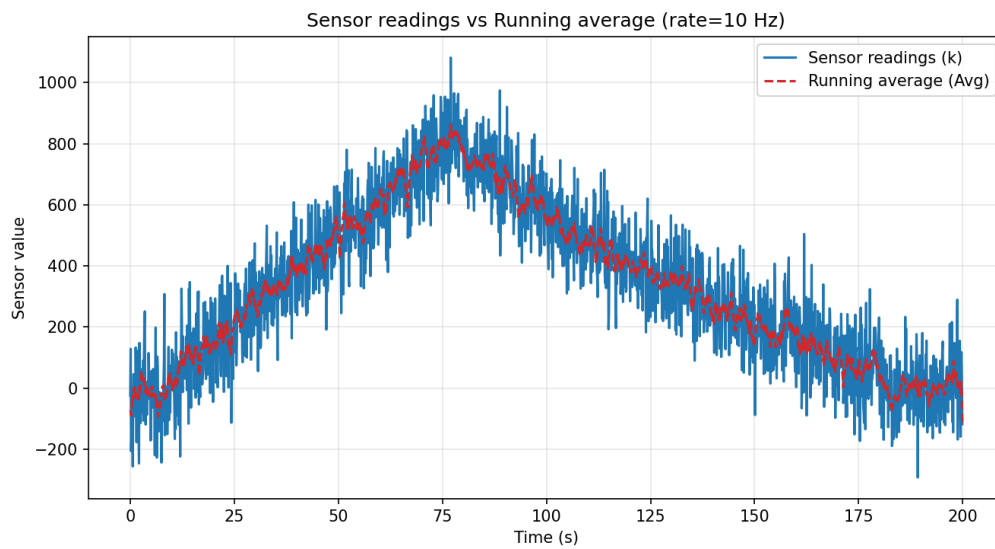
(Sensor noise)
Adding the Gaussian Noise
 $dt['ALT'] = dt['ALT'] + np.random.normal(0, 10, n)$
 $n: len(df)$
mean s.d.
→ Why $\mu = 0$? → So that noise is centered around 0.
→ Why S.D: 10? → To avoid "ideal" noise which is far from reality.

(Pop!)
@ 10 Hz
we take $[x_1, x_2, \dots, x_{10}] \Rightarrow \bar{x}_1$
 $[x_2, x_3, \dots, x_{11}] \Rightarrow \bar{x}_2$
 $[x_{110}, \dots, x_{120}] \Rightarrow \bar{x}_{200}$
Like this we will get 200 sampled values. (~~2000~~ $2000 \div 10 = 200$)
Similarly @ 100 Hz → 20 sample values.

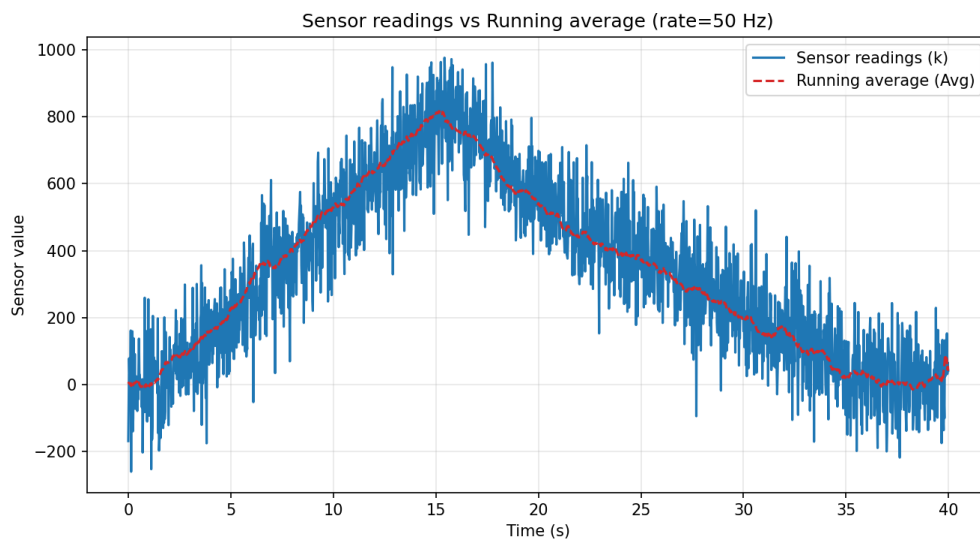
Hence in the results, one might notice that @ 10 Hz avg. filtered values are more dead than @ 100 Hz.

But, in real-life @ 100 Hz will give more accurate values. It's just that 2000 Pkts. is fixed hence we don't have enough data @ 100 Hz to sample.
If we had real incoming values it would have worked as expected.

A. @10 Hz (from A:C, $\sigma = 100$)



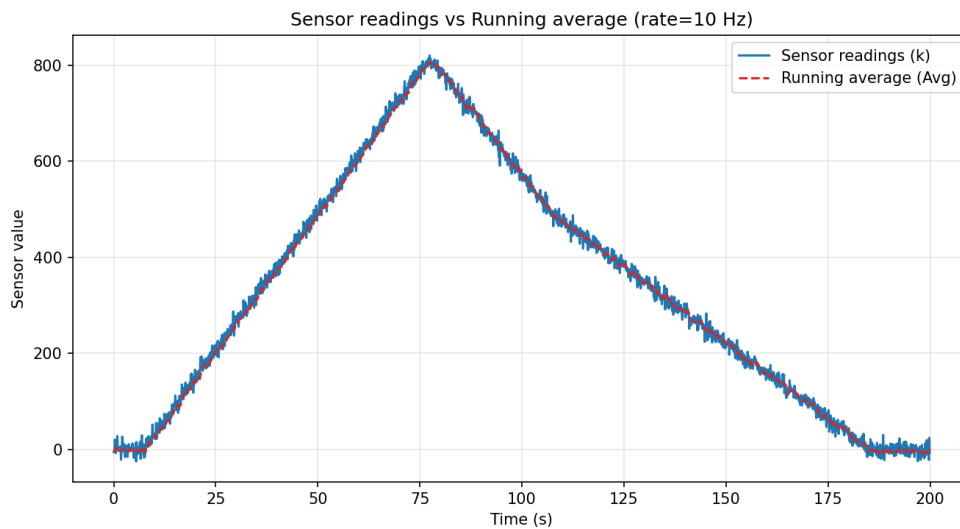
B. @50 Hz



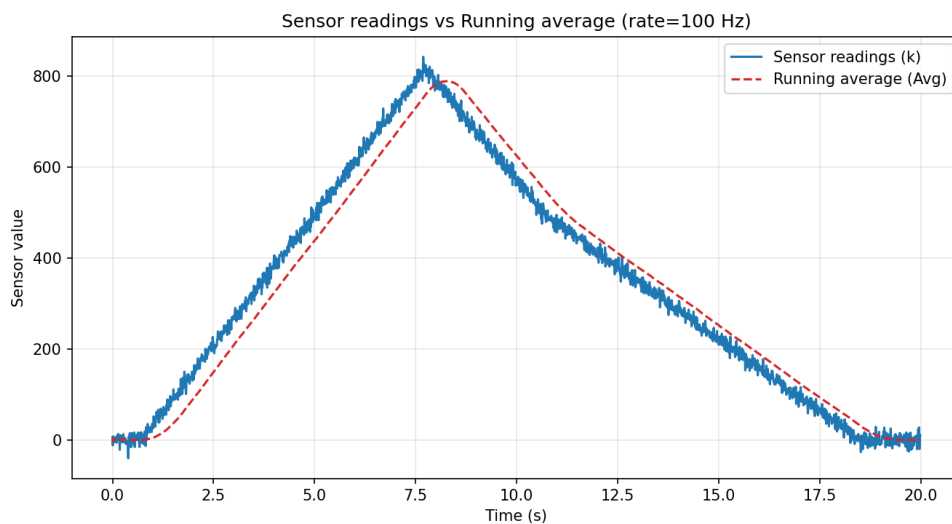
C. @100 Hz



D. @10 Hz



E. @100 Hz



What does this all mean?

Plot A: At 10 Hz, red line is well fit within the actual sensor readings. Apogee area using filter is same as apogee area as raw readings (peak of the triangle). Note the X-axis's readings (200)

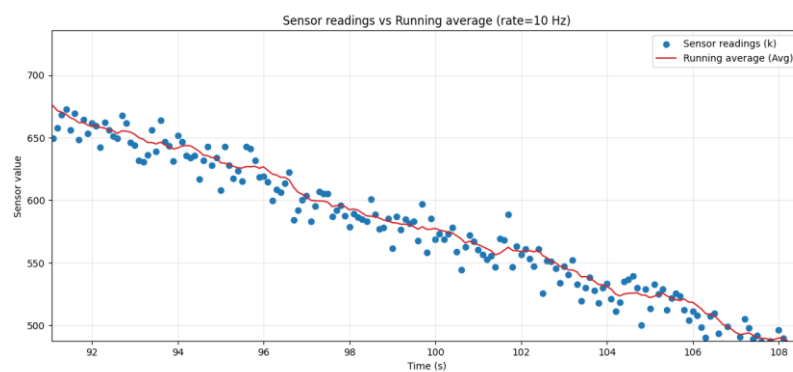
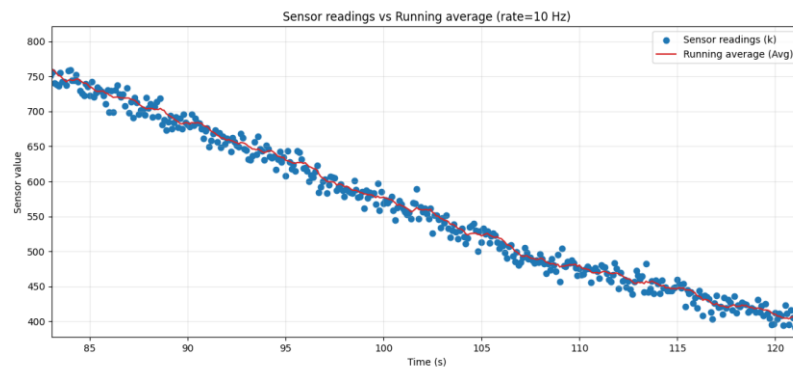
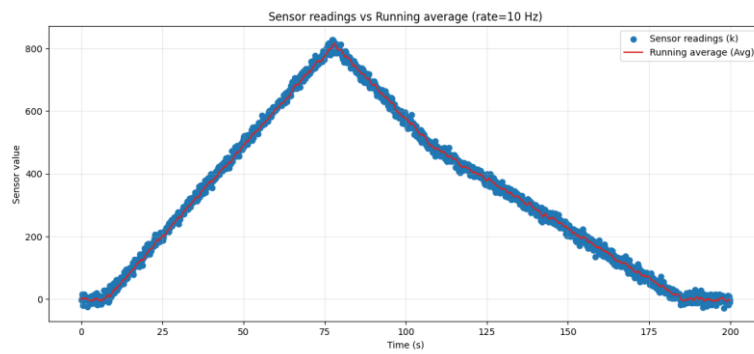
Plot B: At 50 Hz, though there is a deviation of peak from filtered value and actual reading, but still acceptable. Note the X-axis's readings (40). This is what the handwritten note is implying.

Plot C: At 100 Hz, the deviation is significant showing, the absence of enough data to sample at that rate. Note the X-axis's readings (20). Hence as we increase the rate, keeping the csv length fixed, we will keep getting less and less sampled values and eventually that will be not accurate or even near to the real sensor readings.

Plot D: As the SD is reduced to 10 from 100, plot is much cleaner, and same as Plot A, it is very well in accordance to the sensor readings.

Plot E: Even after reducing the SD to 10, @100 Hz, deviation is seen, reason is same as Plot C.

Zoomed-in view of the plot (@10 Hz, SD=10)



Conclusion: The averaging filter works as expected, the only drawback is that the fixed CSV of 2000 pkt. is restricting to demonstrate the oversampling of values. **Oversampling is a method of smoothening (removing noise/spikes) from the sensor readings to get much more accurate values over a period of time. It involves sampling sensors at a much higher rate than required. Less the oversampling rate, poor is the smoothening**

NEXT STEPS:

1. Upgrade Avg. Filter to Low Pass Filter using a fix α for assigning the weight (importance) to most recent value rather than giving all values the same weight.
2. Improving LPF with fix α to dynamic α and applying Kalman filter to get Velocity from Altitude values, without even needing to measure the Velocity using standard equation.